

HW 2: Time Command

Task 1. Implement a time1 command that reports elapsed time.

In this task, my ability to visualize and understand the process of creating processes and checking that they are actually created increased, I also became more comfortable with the idea of taking time and understanding the process of “counting” the CPU time using the uptime call.

```
edward_lopez@edwardsLaptop:~/OS/myxv6$ make qemu
riscv64-linux-gnu-gcc -Wall -Werror -D -fno-omit-frame-pointer -ggdb -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o user/timel.o user/timel.c
riscv64-linux-gnu-ld -z max-page-size=4096 -N -e main -ltext 0 -o user/timel user/timel.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-ld: warning: user/timel has a LOAD segment with RMX permissions
riscv64-linux-gnu-objdump -S user/timel > user/timel.asm
riscv64-linux-gnu-objdump -t user/timel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/timel.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc user/_zombie user/_pipel user/_uptime user/_timel
nmeta 46 (boot, super, log blocks 30 inode blocks 13, bitmap blocks 1) blocks 954 total 1000
ballocc: first 598 blocks have been allocated
ballocc: write bitmap block at sector 45
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ ls
.          1 1 1024
..         1 1 1024
README    2 2 2226
cat        2 3 21152
echo       2 4 20112
forktest   2 5 12144
grep       2 6 23664
init       2 7 20960
kill       2 8 20000
ln         2 9 19912
ls         2 10 23376
mkdir      2 11 20096
rm         2 12 20088
sh         2 13 36128
stressfs   2 14 21088
usertests  2 15 136456
grind      2 16 34064
wc         2 17 21824
zombie     2 18 19520
pipel      2 19 20800
uptime     2 20 19736
timel      2 21 21072
console    3 22 0
$ timel
Error$ uptime
The number of timcks ,60 $ QEMU: Terminated
edward_lopez@edwardsLaptop:~/OS/myxv6$
```

Task 2. Keep track of how much cputime a process has used.

I changed the proc.c file in order to include wait2, in the kernel I created a pstat.h file to include the structure of rusage and then included the structure in user.h as well as creating a system call for wait2 in user.h. I also created an entry in usys.pl and then gave the syscall.h and syscall.c the system call information for wait2. I also modified the syscall.c and proc.c files in the kernel so that they included the return of a child process status and rusage.

Task 3. Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

I implemented syswait2 in the syscall.c and implemented wait2 proc.c files in the kernel so that syswait2 returns the child process status and the rusage status. I also implemented wait2 in

defs.h in order for it to be visible outside of the proc.c file and included “#include pstat.h” so that the kernel can use the structure rusage. Task 2 and Task 3 both taught me a lot more about the kernel and implementation needed on different files in order to get a system call to be implemented. I became much more comfortable with navigating through the user files and kernel files and checking that all of the files have the necessary components to allow someone to create a system call and implement it.

Task 4. Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

```
riscv64-linux-gnu-objdump -S user/_timel > user/timel.asm
riscv64-linux-gnu-objdump -t user/_timel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/timel.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs
user/_usertests user/_grind user/_wc user/_zombie user/_pipel user/_uptime user/_timel user/_matmul user/_sleep
nmeta 46 (boot, super, log blocks 30 inode blocks 13, bitmap blocks 1) blocks 954 total 1000
ballocc: first 641 blocks have been allocated
ballocc: write bitmap block at sector 45
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

lsinit: starting sh
$ ls
exec ls failed
$ ls
.          1 1 1024
..         1 1 1024
README    2 2 2226
cat        2 3 21152
echo       2 4 20112
forktest   2 5 12144
grep       2 6 23664
init       2 7 20960
kill       2 8 20000
ln         2 9 19912
ls         2 10 23376
mkdir      2 11 20096
rm         2 12 20088
sh         2 13 36120
stressfs   2 14 21008
usertests  2 15 136456
grind      2 16 34064
wc         2 17 21824
zombie     2 18 19520
pipel      2 19 20880
uptime     2 20 19736
timel      2 21 21072
matmul     2 22 20656
sleep      2 23 19824
console    3 24 0
$ time matmul
Time: 17 ticks
elapsed time: 17 ticks, cpu time: 17 ticks, 100% CPU$ QEMU: Terminated
edward_lopez@edwardsLaptop:~/OS/myxv6$ cd user
edward_lopez@edwardsLaptop:~/OS/myxv6/user$ cd ..
edward_lopez@edwardsLaptop:~/OS/myxv6$
```

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).